

操作系统原理

第四章：进程管理

洪明坚

重庆大学软件学院

February 19, 2019

- 1 Process
 - Process and Program
 - Process state
 - Process control block
- 2 Process scheduling
 - Context switch
 - Scheduling queues
 - The scheduler
- 3 Operations on process
 - Creation
 - Termination
- 4 Cooperating processes
 - The producer and consumer
- 5 Interprocess communication (IPC)
 - Pipe

Outline

- 1 Process
 - Process and Program
 - Process state
 - Process control block
- 2 Process scheduling
 - Context switch
 - Scheduling queues
 - The scheduler
- 3 Operations on process
 - Creation
 - Termination
- 4 Cooperating processes
 - The producer and consumer
- 5 Interprocess communication (IPC)
 - Pipe

Process concept (1/3)

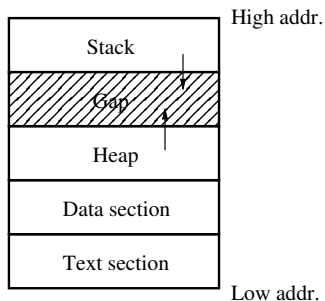
- Before the advent of process
 - Batch system - **job**;
 - Multiprogramming or time-sharing - **program or task**.
- **Process**
 - **An abstraction of a running job/program/task.**

Process concept (2/3)

- What's a process?
 - A program in execution.
- A process is MUCH more than a program. It consists of
 - **Text section** (executable machine codes) from the program file;
 - **Data section** (the global variables) from the program file;
 - *Contents* of the processor's **registers**;
 - **Stack** which contains temporary data such as function parameters, return addresses and local variables;
 - **Heap** which is the memory used to be allocated dynamically if necessary;
 - A lot of other resources such as open files, etc.

Process concept (3/3)

- **Process image** in memory



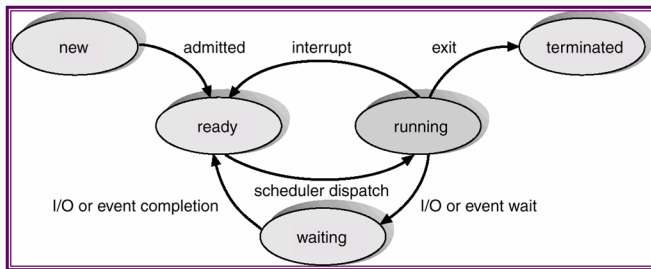
Program v.s. process

- The program is a **passive** entity which resides in some storage;
 - While the process is an **active** entity which contains a lot of resources other than the program.
- Many processes may be running the same program. But,
 - They are considered as separate execution sequences although they share the same text section;
 - Other resources usually vary.

- As a process executes, it changes **state**.
 - The state of the process is defined by its current activity.
- Each process may be in one of the following states:
 - **New** - The process is being created;
 - **Running** - Instructions are being executed;
 - **Waiting** - The process is waiting for some event to occur;
 - **Ready** - The process is waiting to be assigned to a processor;
 - **Terminated** - The process has finished its execution.

State transition (1/2)

- As the process proceeds, it will transit from current state to another.
 - There are six transitions among these five states.



State transition (2/2)

- Transitions

Admitted The process has been created and is ready to run;

Scheduler dispatch The **scheduler** picks the process to run;

Interrupt An interrupt has occurred;

I/O event wait The process blocks for I/O completion or reception of an signal;

I/O event completion I/O has completed or an event has occurred;

Exit The process has finished its execution.

Process Control Block

- Each process is represented in the operating system by a **Process Control Block (PCB)**.
 - Also known as **Task Control Block**.
- Notes
 - ❶ **Process number** is an unique identifier of a process, also known as **PID**.
 - ❷ **Program counter (PC)** is one of registers.
 - ❸ **Scheduling information** includes process' priority, etc.

process #
process state
program counter
registers save area
memory management information
accounting info
I/O status info.
scheduling info.

Questions

- Any questions?



Outline

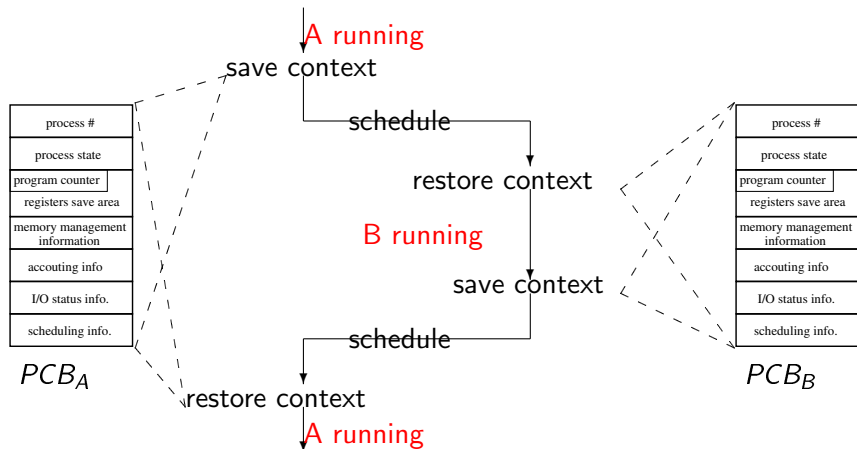
- 1 Process
 - Process and Program
 - Process state
 - Process control block
- 2 Process scheduling
 - Context switch
 - Scheduling queues
 - The scheduler
- 3 Operations on process
 - Creation
 - Termination
- 4 Cooperating processes
 - The producer and consumer
- 5 Interprocess communication (IPC)
 - Pipe

Process scheduling

- When running process can not proceed for some reason, the operating system must **decide which process to run next**.
 - Three main concerns:
 - ① What happens to the process currently running?
 - ② How to keep track of what each process should be doing?
 - ③ How to decide which process to run next?

Concern 1

- What happens to the process currently running?
 - Context switch**

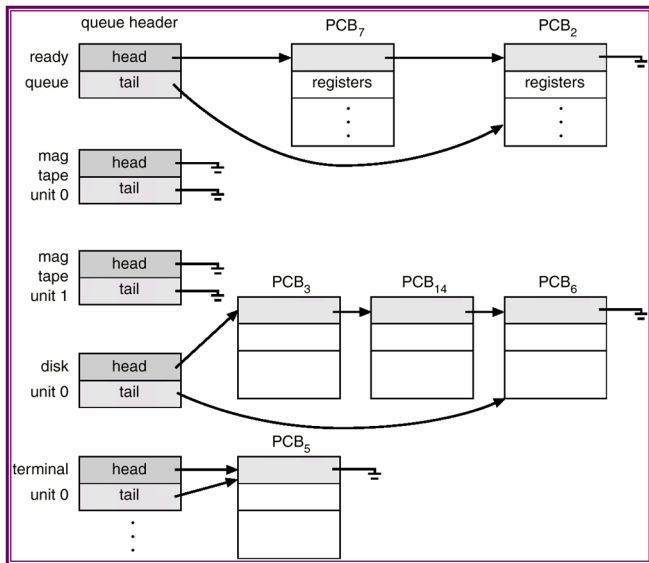


Context switch

- Context switch is **pure** overhead.
 - Depending on the underlying processor, it may burn **a lot of** CPU cycles.
 - Context switching has become such a performance bottleneck that programmers are using a new structure to avoid it whenever possible.

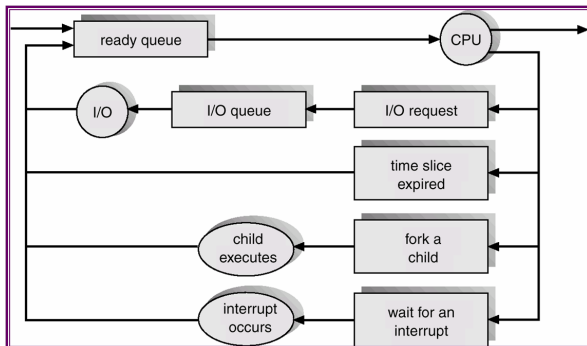
- How to keep track of what each process should be doing?
 - **Scheduling queues**
 - **Job queue** - consists of all processes in the system;
 - **Ready queue** - consists of processes waiting for CPU to execute;
 - The operating system also has other queues.
 - **I/O device queue** - consists of processes waiting for a particular I/O device.

Various I/O device queues



Queuing diagram

- The queuing diagram is a common representation of process scheduling in the operating system.



Concern 3

- How to decide which process to run next?
 - It's the **scheduler** which makes this decision.
- The scheduler selects a process from the ready queue and allocates CPU to it.
 - We call this scheduler the **CPU scheduler**.

Questions

- Any questions?



Outline

- 1 Process
 - Process and Program
 - Process state
 - Process control block
- 2 Process scheduling
 - Context switch
 - Scheduling queues
 - The scheduler
- 3 Operations on process
 - Creation
 - Termination
- 4 Cooperating processes
 - The producer and consumer
- 5 Interprocess communication (IPC)
 - Pipe

Operations on process (1/2)

- Creation - a “parent” process spawns a “child” process.
 - Parent process create children processes, which, in turn create other processes, forming a tree of processes.
 - For example, *fork* in Unix/Linux and *CreateProcess* in Windows.

```
int main() {  
    int pid = fork(); /*fork another process*/  
    if(pid < 0) {      /*error occurred*/  
        printf("fork failed\n");  
        exit(-1);  
    } else if (pid == 0) /*in child process*/  
        execlp("/bin/ls", "ls", NULL); /*executes a program*/  
    else { /*in parent process*/  
        waitpid(pid, NULL, 0); /*waiting for the child to exit  
                                */  
        printf("Child exited\n");  
        exit(0);  
    }  
}
```

Operations on process (2/2)

- Termination. Two kinds of termination:
 - normal** a process asks the operating system to delete it. For example, *exit* in Unix/Linux and *ExitProcess* in Windows.
 - abnormal** one process may be killed by another process by *kill* in Unix/Linux and *TerminateProcess* in Windows.
- Communication - facilities used by one process to communicate with another.
- Cooperation - cooperating processes need to synchronize their actions.

Questions

- Any questions?



Outline

- 1 Process
 - Process and Program
 - Process state
 - Process control block
- 2 Process scheduling
 - Context switch
 - Scheduling queues
 - The scheduler
- 3 Operations on process
 - Creation
 - Termination
- 4 Cooperating processes
 - The producer and consumer
- 5 Interprocess communication (IPC)
 - Pipe

Cooperating processes

- A process can be
 - **independent** if it cannot affect or be affected by the other processes;
 - **cooperating** otherwise.
- There are several advantages for the process cooperation.
 - Information sharing:** Several processes may be interested in the same piece of information;
 - Computation speedup:** Break the problem into several subtasks that can be run in parallel;
 - Modularity:** Separate processes for different functions by design;
- However, concurrent execution of cooperating processes requires mechanisms that allow processes to communicate with one another and to synchronize their actions.

- **The producer and consumer problem**
 - A **producer** process produces information that is consumed by a **consumer** process.
 - This problem is a common paradigm for cooperating system.
- To allow producer and consumer processes to run concurrently, we need
 - a buffer which can be filled by the producer and emptied by the consumer;
 - to synchronize the producer and consumer.
- According the size of the buffer, we have
 - **unbounded-buffer** producer and consumer problem;
 - **bounded-buffer** producer and consumer problem.

Implementation

- A solution to **bounded-buffer producer and consumer problem**.

```
/* Shared variables */
#define BSIZE 10
struct item {
    ....
} buffer[BSIZE];
int in = 0, out = 0;

/*-----*/
/*The producer loop*/
while(1) {
    /*produce an item*/
    while(((in + 1) % BSIZE) == out)
        /*do nothing*/;
    buffer[in] = itemProduced;
    in = (in + 1) % BSIZE;
}

/*The consumer loop*/
while(1) {
    while(in == out)
        /*do nothing*/;
    itemConsumed = buffer[out];
    out = (out + 1) % BSIZE;
    /*consume the item*/
}
```

Questions

- Any questions?



Outline

- 1 Process
 - Process and Program
 - Process state
 - Process control block
- 2 Process scheduling
 - Context switch
 - Scheduling queues
 - The scheduler
- 3 Operations on process
 - Creation
 - Termination
- 4 Cooperating processes
 - The producer and consumer
- 5 Interprocess communication (IPC)
 - Pipe

Interprocess communication (IPC)

- What's the IPC?
 - **IPC provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space.**
- Examples
 - Message-passing, pipe, socket, etc.

Pipe (1/3)

- **Pipe** - a FIFO communication channel between two processes.
 - A pipe has two ends: one for reading from it and the other for writing to it.
- There are two kinds of pipe:
 - Anonymous pipe
 - An unnamed, simplex (one-way) channel used to transfer data between the parent and child processes.
 - It exists for as long as the process is running.
 - Named pipe
 - As an extension of unnamed pipe, it can be simplex or full-duplex and may be used by any process to communicate with each other.
 - It can be permanent (as in Unix/Linux) or volatile (as in Windows).

Pipe (2/3)

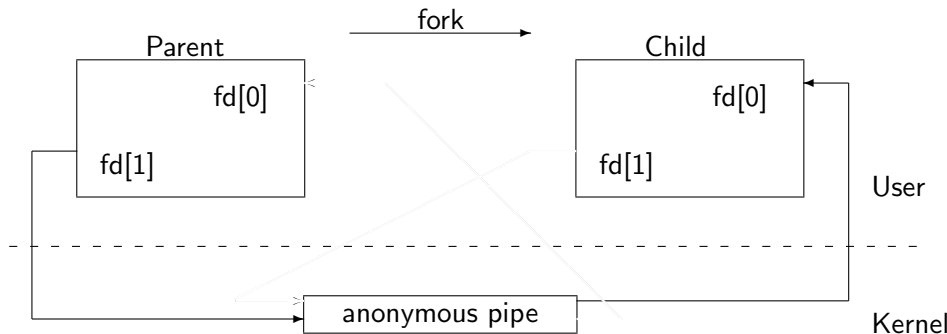
```
int main() {
    int fd[2], pid;    char buf;

    if (pipe(fd) == -1) /* Create an anonymous pipe */
    { perror("pipe"); exit(-1); }
    if ((pid = fork()) < 0)
    { perror("fork"); exit(-1); }
    if (pid == 0) {      /* Child reads from pipe */
        close(fd[1]);    /* Close unused write end */

        while (read(fd[0], &buf, 1) > 0)
            write(STDOUT_FILENO, &buf, 1);

        close(fd[0]);
        exit(0);
    } else {             /* Parent writes to pipe */
        close(fd[0]);    /* Close unused read end */
        write(fd[1], "Hello, child!", 13);
        close(fd[1]);    /* Reader will see EOF */
        waitpid(pid, NULL, 0); /* Wait for child to
                                exit*/
        exit(0);
    }
}
```

Pipe (3/3)



Questions

- Any questions?

